

Subagentes: contexto limpo e custo sob controle

Pesquisa sobre como os subagentes do Claude Code evitam a "degradação de contexto" em sessões longas e, se bem usados, reduzem em vez de multiplicar o custo em tokens — feita a partir da prática "O Arquiteto de Agentes e a Tech Lead Cansada de o Claude Amolecer".

Fernando Melo · 8 de setembro de 2026

01 · O PROBLEMA

Por que o Claude "amolece" em sessões longas

Em sessões únicas e longas, o modelo começa a se perder, ignorar instruções do system prompt e fugir dos padrões definidos — o chamado "efeito de degradação do contexto". Isso não acontece pela complexidade do código em si, mas pelo tamanho do histórico de conversa: quanto mais mensagens de erro, tentativa e correção se acumulam, mais o modelo se confunde sobre qual instrução ainda vale.

Subagentes não servem para "rodar vários Claudes ao mesmo tempo" na mesma tarefa — servem para isolar contexto e responsabilidade. Cada subagente é melhor entendido como uma função de código isolada, gerenciada automaticamente pelo agente principal, e não como um chat paralelo aberto à parte.

02 · SESSÃO ÚNICA VS. SUBAGENTES

O que muda na prática

ASPECTO	SESSÃO ÚNICA LONGA	USO DE SUBAGENTES
Tamanho do contexto	Acumula tudo (erros, testes, decisões antigas); o Claude fica lento e "esquecido"	Curto e cirúrgico — cada subagente recebe só o necessário para sua microtarefa
Adesão a padrões	Perde o foco conforme o histórico cresce; começa a alucinar ou ignorar regras	Alta — histórico limpo, segue o system prompt e o guia de estilo à risca
Custo de tokens	Cresce a longo prazo, pois cada mensagem reenvia todo o histórico	Controlado — paga-se só pelo contexto da microtarefa

O sistema de arquivos como ponto de sincronização

Os contextos não ficam soltos porque quem sincroniza o trabalho não é a memória do Claude, e sim o próprio código do projeto no disco. O fluxo funciona como uma pequena empresa:

- 1 Agente principal (arquiteto) — mantém a visão macro e as decisões de arquitetura, documentadas em arquivos locais (ex.: um README de arquitetura ou `.claudecode/instructions`).
- 2 Subagentes (operários) — o principal abre um subagente com instrução fechada: "pegue a decisão X no arquivo Y, implemente o teste da função Z e encerre".
- 3 Fechamento — ao terminar, a sessão do subagente é destruída; os tokens daquela discussão somem, mas o código e o progresso ficam salvos no repositório.

Quando um segundo subagente é criado na sequência, ele não precisa saber o que o primeiro conversou — só precisa ler o arquivo já atualizado no disco. O código é a única verdade que importa; o agente principal apenas monitora o estado dos arquivos e a lista de metas.

Por que dividir em subagentes pode custar menos

A cobrança dos LLMs é acumulativa: paga-se pelo histórico inteiro a cada nova mensagem. Considerando um contexto base fixo de 10.000 tokens:

Cenário A — sessão única

Na mensagem 31, o histórico já acumulou +30.000 tokens de conversas, erros e tentativas passadas. Corrigir um simples ponto e vírgula custa mais de 40.000 tokens de entrada. O custo cresce como bola de neve.

Cenário B — subagentes

O agente principal mantém só os 10.000 tokens de base. Um subagente nasce com esse contexto + o arquivo necessário, resolve em 1-2 mensagens (~11.000 tokens) e é deletado. Os 30.000 tokens de "conversa fiada" nunca chegam a existir para o histórico principal.

Ponto de virada: subagentes começam a economizar a partir do momento em que a sessão única passaria de 3 a 4 interações longas. Se a tarefa exige muitas idas e vindas (erros de teste, ajustes de sintaxe), delegar é drasticamente mais barato do que resolver no chat principal.

Onde está o perigo: se o agente principal repassar contexto "cego" ao criar o subagente — projeto inteiro, logs imensos, arquivos de mídia desnecessários — paga-se o custo de inicialização de uma janela gigante todas as vezes. O segredo da economia é a "dieta de contexto".

Como usar subagentes sem explodir a conta

- 1 Aproveite o Context Caching:** a Anthropic dá desconto de até 90% em tokens que ficam em cache. Mantendo fixos os arquivos de regras estruturais (`.claudecode/instructions`, docs de arquitetura), o subagente os lê do cache por uma fração do preço.
- 2 Restrinja o escopo por comando:** em vez de "ajuste o módulo de usuários", diga "altere apenas `src/users/controller.ts` para adicionar a validação do campo X". Menos arquivos lidos = menos tokens faturados.
- 3 Limite as iterações:** não deixe o subagente girar em loop tentando corrigir um erro sozinho; se falhar 2 ou 3 vezes, mate a sessão e assuma o controle.

Não use subagente para

ajustes rápidos, perguntas conceituais, correções de sintaxe simples ou tarefas diretas que o agente principal resolve de primeira.

Use subagente para

tarefas pesadas e iterativas com alto potencial de gerar "lixo" no histórico: debugar loops de erro em testes, refatorações em lote, revisões profundas de segurança ou escrita de boilerplate massivo.

Papéis fixos, salvos como arquivo no repositório

No Claude Code, isso não exige um sistema complexo: basta moldar o comportamento com arquivos de instruções locais (ex.: pasta `.claudecode/`). Por ficarem salvos como Markdown dentro do repositório, qualquer desenvolvedor do time que rodar o Claude Code no projeto tem acesso instantâneo aos mesmos "assistentes especialistas" — padronizando revisão de segurança, escrita de testes e estilo de código antes mesmo de abrir um Pull Request.

Revisor de segurança

`.claudecode/security-agent.md` — analisa o arquivo indicado em busca de SQL Injection, XSS, falhas de autenticação e vazamento de segredos, sem alterar lógica de negócio. Uso: `@security-agent revise as alterações em src/auth/login.ts` antes do commit.

Criador de testes

`.claudecode/test-agent.md` — lê o código-fonte e gera o teste equivalente em Jest/Vitest, buscando cobertura de edge cases e rodando `npm test`. Uso: `@test-agent crie os testes unitários para a função de pagamento em src/services/payment.ts`.

Um subagente especializado tende a ser mais preciso porque seu system prompt é curto e direto (não divide atenção entre várias funções) e não carrega o histórico de decisões de negócio mudando ao longo do dia — aplicando os padrões de forma mais estrita e "fria".

O subagente de segurança para Node.js + PostgreSQL + Passport.js

Na prática, o primeiro subagente escolhido foi o revisor de segurança (AppSec) — cenário clássico de foco cirúrgico e de longos debates que poluíam a sessão principal. O arquivo final, ajustado para a stack PostgreSQL + Passport.js, ficou assim:

Conteúdo integral de .claudecode/security-agent.md, conforme gerado na prática e confirmado na transcrição oficial.

```
## Role: Subagente Especialista em AppSec (Node.js + PostgreSQL + Passport.js)
```

```
Você é um subagente focado estritamente em segurança de código (Application Security).
Sua missão é analisar o arquivo fornecido pelo usuário e identificar vulnerabilidades.
```

```
## Escopo de Atuação (Foco Total)
```

1. Analise APENAS o arquivo indicado. Não tente ler ou alterar outros arquivos a menos que seja estritamente necessário.
2. Não altere regras de negócio, formatação de código ou estilos de escrita se não houver um risco de segurança associado.
3. Se encontrar vulnerabilidades, corrija-as diretamente no arquivo, comentando brevemente a linha alterada.

```
## Checks Específicos da Stack
```

```
### 1. PostgreSQL (Segurança de Dados)
```

- SQL Injection: nenhuma query com concatenação direta de strings; exigir queries parametrizadas ou ORMs seguros (Knex, Prisma).
- Tratamento de erros: erros nativos do Postgres não podem ser expostos direto na resposta da API.

```
### 2. Passport.js (Autenticação e Sessões)
```

- Validar dados extraídos das estratégias (ex.: req.user) antes de usá-los em lógica crítica ou queries.
- serializeUser/deserializeUser não devem trafegar dados sensíveis (ex.: hash de senha) na sessão.
- JWT com expiresIn curto; segredo sempre via process.env, nunca hardcoded.

```
### 3. OWASP Top 10 (geral)
```

- Falhas de controle de acesso (BOLA), XSS, segredos vazados no código.

```
## Output Esperado
```

```
Se seguro: "Arquivo analisado e nenhum risco crítico encontrado."
```

```
Se houver problema: corrigir direto no código e resumir em tópicos curtos de uma linha.
```

Como usar no dia a dia:

- 1 Salvar o conteúdo em .claudecode/security-agent.md, dentro do projeto.
- 2 Ao criar um endpoint crítico (login, transações), chamar o subagente direto pelo terminal em vez de pedir ao agente principal: claude "leia as instruções em .claudecode/security-agent.md e revise apenas o arquivo src/api/payment.ts".
- 3 O subagente abre, aplica a mentalidade de um hacker ético, reescreve as vulnerabilidades direto no arquivo se necessário, e fecha — só custando os tokens estritamente necessários para aquela varredura.

Dica de ouro para o primeiro teste: escrever de propósito, num arquivo isolado, uma query vulnerável por concatenação de strings (ex.: `client.query("SELECT * FROM users WHERE email = '" + req.body.email + "'" )`) e mandar o subagente revisar. O esperado é ver a query virar sintaxe parametrizada segura, o arquivo ser salvo e a sessão encerrar sem gastar tokens extras de histórico.